# ANALYSIS

# HXC-169 — Container-sandbox advisory reachability determination

**Revision:** 1 **Date:** 2026-07-31 **Status:** Resolved — UNREACHABLE IN OUR CONFIGURATION **Guard:** `scripts/gates/openhands_docker_sandbox_unwired_gate.sh` (`CM-OPENHANDS-DOCKER-SANDBOX-UNWIRED`), committed `95c27998`

---

## Why this document exists

The gate committed in `95c27998` cites this path in two places, including the remediation text it prints on failure. The agent that produced the determination could not write it: its harness blocks subagents from authoring analysis `.md` files. It flagged the conflict and the resulting dangling reference rather than routing around the guard, which is the correct behaviour — so the doc is written here, from that agent's reported evidence plus the claims independently re-run and named as such below.

---

## Verdict

The five advisories affecting `github.com/docker/docker` are **not reachable in any binary this project ships**. The sandbox that calls the affected operations exists, compiles, and is genuinely vulnerable — and **nothing imports it**.

This is *not* "the risk is low". It is a structural claim: the package is absent from the link graph of every `cmd/`, so the vulnerable code is never present in a shipped artifact at all.

## Reconciling the apparent contradiction

The tracker item states an automated analysis DID trace calls from our sandbox code to the affected functions. A predecessor agent found ZERO importers of that sandbox code. Both are true simultaneously, and the reconciliation IS the answer:

> The sandbox code calls the vulnerable functions. Nothing calls the sandbox code.

The root cause of the confusion is mechanical. The originating sweep (HXC-166) ran `govulncheck ./...`, which treats **every package as an analysis root** — including packages no binary imports. Scoped correctly, the result inverts:

| Scope | Result |
|---|---|
| `govulncheck ./internal/clis/openhands/...` | **exit 3** — traces `sandbox.go:298 CopyTo → CopyToContainer`, `sandbox.go:312 CopyFrom → CopyFromContainer` |
| `govulncheck ./cmd/...` | **exit 0** — "No vulnerabilities found", zero `docker/docker` mentions even in verbose |

# Independently re-verified (by the orchestrator, not taken on report)

| Claim | Command | Result |
|---|---|---|
| Binaries carry no docker dependency | `go list -deps ./cmd/...` in `submodules/helix_agent` | 1097 deps, **0** `github.com/docker/docker`, **0** `clis/openhands` |
| The wired sibling is a different package | read `internal/clis/agents/master/master.go:19` | imports `dev.helix.agent/internal/clis/agents/openhands` — NOT `internal/clis/openhands` |
| Never wired, ever | `git log --all -S 'clis/openhands"'` | zero commits in all history |

# The trap that makes this easy to get wrong

**Two packages share the name openhands:**

- `internal/clis/openhands` — the Docker sandbox. Imports `docker/docker`. **Unwired.**
- `internal/clis/agents/openhands` — **wired** via `agents/master/master.go:19,48`. Uses `os/exec` to drive the external OpenHands CLI binary. No Docker import; it mentions `"docker"` only as a `SandboxType` config *string*.

A name-based search conflates them and concludes the sandbox is live. Only the import graph separates them.

## Hidden-reference mechanisms ruled out (§11.4.124)

“Zero importers ⇒ unreachable” is a **guess** until each hidden-reference path is excluded. Per the reporting agent:

| Mechanism | Ruled out by |
|---|---|
| Build tags | No `//go:build` in `sandbox.go`; the scan is text-based so it sees files any build tag would exclude — zero hits |
| `init()` side effects | No `func init()` in the package; and `init` runs only if linked, which requires an import |
| Reflection / dynamic dispatch | Go reflection cannot instantiate a type from a package not linked into the binary — structural, not empirical |
| Code generation | No `go:generate` referencing openhands/sandbox |
| Go `plugin` loading | Nothing imports stdlib `"plugin"`; matches are the *string* `"plugin"` in config key-sets |
| Name-based registry / DI | The `openhands` strings elsewhere name the external CLI binary and resolve to the wired sibling |
| Test-only wiring | `TestImports` and `XTestImports` both empty |
| helix_code importing it | **Compiler-enforced impossible** — `dev.helix.agent/internal/...` is importable only within `dev.helix.agent/...`; helix_code is module `dev.helix.code` |

## The advisories — five, not four

The tracker item and HXC-166’s prose say four; HXC-166’s own table lists five. Fetched live 2026-07-31 from GitHub Advisory DB + OSV + pkg.go.dev/vuln (§11.4.99):

| GHSA | CVE | Severity | Affected | Fixed in | Layer |
|---|---|---|---|---|---|
| GHSA-rg2x-37c3-w2rh | CVE-2026-42306 | High 7.2 | ≤ 28.5.2 | **none** | daemon |
| GHSA-x86f-5xw2-fm2r | CVE-2026-41567 | High 7.2 | ≤ 28.5.2 | **none** | daemon |
| GHSA-x744-4wpc-v9h2 | CVE-2026-34040 | High 8.8 | < 29.3.1 | 29.3.1 | daemon |
| GHSA-vp62-88p7-qqf5 | CVE-2026-41568 | Moderate 6.1 | ≤ 28.5.2 | **none** | daemon |
| GHSA-pxq6-2prw-chj9 | CVE-2026-33997 | Moderate 6.8 | < 29.3.1 | 29.3.1 | daemon |

All five list our pin as affected. The three without a fix are patched only on the separate, still-beta `moby/moby/v2@2.0.0-beta.14`.

**Material nuance:** all five are **daemon-side**, and the two carrying Go symbol data name unexported daemon methods we never import (`Daemon.openContainerFS`, `Daemon.containerExtractToDir`). Client-side use is not irrelevant — a client copy is what *opens* the daemon's TOCTOU window — which is exactly why "do we ever issue those copies?" was the right question to ask.

# Correction to the tracker

The code lives in **`submodules/helix_agent`**, not helix_code. HXC-166's paths are relative to that submodule. helix_code's own `go.mod`/`go.sum` contain **zero** docker references.

# Why it is unwired (git history, per §11.4.124)

`git log --all -S 'clis/openhands"'` returns **zero commits across all history**: the package was never wired at any point. A port landed in `a5f2857d` and was never connected. This is **never-completed wiring**, not a deleted call site and not a regression. Under §11.4.124 that means it is **not** deleted on sight — removal of a shipped capability additionally requires operator sign-off (§11.4.122).

## Why no containment change was implemented

- *Disable the feature* — not applicable; nothing enables it. There is no env var, config key, feature flag, or build tag that constructs a sandbox. It requires a Go caller, and none exists. The "deciding file" is the **absence of an importer**.
- *Constrain the copy paths* — would add unexercised, unvalidatable code: a bluff surface rather than a mitigation.
- *Isolate the container socket* — concerns different subsystems (see open item below).

## The guard, and why an unreachability claim needs one

An unreachability verdict with no guard rots silently the moment someone wires the package up. `CM-OPENHANDS-DOCKER-SANDBOX-UNWIRED` asserts:

- **(A)** no `.go` file outside the package imports it — a text scan, deliberately build-tag-independent, with a trailing quote to exclude the wired sibling; honest §11.4.3 SKIP if the submodule is absent;
- **(B)** `github.com/docker/docker` stays out of helix_code's module files.

Paired §1.1 self-test against hermetic temp fixtures (never writes inside a repo, §11.4.84): **golden-good = 0, golden-bad = 1**. Real run: 5 assertions, 0 violations. On failure it instructs reopening HXC-169 and forbids weakening the assertion (§11.4.120).

## Open — requires operator decision

1. **Finish the sandbox, or drop it?**
   - *Keep as-is* — zero risk today, but ~450 lines of unexercised code keep five advisories appearing in every future scan.
   - *Wire it up* — makes all five genuinely live, three of them with no fix available.
   - *Remove it* — ends the scan noise and drops the dependency, but needs a §11.4.124 separate removal commit and §11.4.90 `Obsolete / feature-removed` classification.
2. **Four read-write `/var/run/docker.sock` mounts** (`docker-compose.helix.yml:39`, `docker-compose.test.yml:202`, `helix_code/docker-compose.builder.yml:30`, `helix_code/docker/docker-compose.yml:369`) plus two `:ro`. **UNCONFIRMED, not cleared.** Not what these advisories turn on, and none of those services runs this sandbox — tracked separately rather than used to close HXC-169.

## Honest limits (§11.4.6)

- This is a **source-and-link-graph** determination, not a runtime one. It says nothing about a human running `docker cp` by hand.
- Advisory state is a **2026-07-31 snapshot**. Re-verify before **2026-10-29** (§11.4.99 90-day bound for risk-classified sources).
- Toolchain runs used default `linux/amd64`. The guard's text scan is platform-independent; the `go list` / `govulncheck` runs are not.
- The socket mounts above are **UNCONFIRMED, not cleared**.
- This speaks only to the Rank-1 docker cluster; the full HXC-166 sweep was not re-run.
- `helix_agent` was read-only throughout the determination. It carried other agents' concurrent edits, none of which any assertion depends on.

## Sources verified 2026-07-31

- GitHub Advisory Database — https://github.com/advisories (per-GHSA pages listed above)
- OSV — https://osv.dev
- Go vulnerability database — https://pkg.go.dev/vuln